



RIOC 2025

TD – Usine de Trombone

Alexandre Létocart

Contexte : Vous construisez une API backend de Clipper, une simulation d'usine qui fabrique des trombones.

Stack : Java 25, Spring Boot 4.0.6, MySQL 8.4 (Docker), Lombok 1.18.46, MapStruct 1.6.3

PS : Le code (noms de classes, variables, endpoints) doit être rédigé en anglais.

1. Création de l'entité Usine

Champs attendus

CHAMP	TYPE	NOTES
id	Long	Clé primaire, auto-générée (IDENTITY)
name	String	Nom de l'usine
production	Integer	Nombre de trombones produits par cycle
stock	Integer	Stock actuel, défaut à 0
latitude	Double	Position géographique (utilisé au TD3)
longitude	Double	Position géographique (utilisé au TD3)

Vous êtes libre d'ajouter des champs.

2. Les différents endpoints

Vous devez livrer plusieurs endpoints :
Chaque endpoint doit retourner le code HTTP approprié.

MÉTHODE	URL	DESCRIPTION
POST	/api/factories	Créer une usine
GET	/api/factories	Lister toutes les usines
GET	/api/factories/{id}	Détail d'une usine
PUT	/api/factories/{id}	Modifier entièrement (remplacement)
PATCH	/api/factories/{id}	Modifier partiellement (champs non-null)
POST	/api/factories/{id}/produce	Produire manuellement (?quantity=N)
DEL	/api/factories/{id}	Supprimer une usine

3. Production automatique des trombones

Après avoir fait le CRUD (*Create, Read, Update, Delete*), chaque usine créer doit produire automatiquement des trombones toutes les 5 secondes. À chaque cycle, le stock de chaque usine augmente de son rythme de production. Cette production doit tourner en arrière-plan sans intervention du client.

Voici des outils qui peuvent servir pour cette automatisation :

- <https://www.baeldung.com/spring-scheduled-tasks>
- <https://www.baeldung.com/spring-scheduling-annotations>



RIOC 2025

TD – Magasin de Trombone

Alexandre Létocart

Contexte : Les usines produisent des trombones, il faut maintenant les acheminer vers des magasins. Un magasin peut commander un réapprovisionnement à une usine. Le transfert n'est pas instantané : les trombones voyagent à la vitesse du son (1235 km/h).

Prérequis : TD1 terminé.

1. Création de l'entité Magasin

Champs attendus

CHAMP	TYPE	NOTES
id	Long	Clé primaire, auto-générée (IDENTITY)
name	String	Nom du magasin
stock	Integer	Stock actuel, défaut à 0
latitude	Double	Position géographique (utilisé au TD3)
longitude	Double	Position géographique (utilisé au TD3)

Vous êtes libre d'ajouter des champs.

2. Réapprovisionnement

Un magasin peut consulter les usines disponibles et commander des trombones.

MÉTHODE	URL	DESCRIPTION
GET	/api/stores/{id}/factories	Lister les usines disponibles avec leur distance
POST	/api/stores/{id}/supply	Commander un réapprovisionnement
GET	/api/stores/{id}/shipments	Lister les expéditions (?status=delivered pour les terminées, en cours par défaut)

Règles :

- **GET** /*factories* retourne toutes les usines avec leur stock et une distance aléatoire (10–1000 km). La distance réelle sera calculée au TD3 via une API externe
- **POST** /*supply* reçoit un identifiant d'usine et une quantité. La réponse doit inclure les informations de localisation (nom, latitude, longitude) de l'usine et du magasin concernés
- Le stock de l'usine est décrémenté immédiatement
- Le stock du magasin augmente uniquement après le temps de livraison
- Si l'usine n'a pas assez de stock → erreur appropriée
- Temps de trajet = $(\text{distance} / 1235) \times 3600$ secondes
- **GET** /*shipments* retourne par défaut les expéditions en cours. Le query param ?status=delivered permet de consulter les expéditions terminées

Vous pouvez ajouter d'autres règles métier

3. Livraison asynchrone

Vous devez modéliser l'expédition en base (usine source, magasin destinataire, quantité, distance, statut, heures de départ/arrivée). Un processus en arrière-plan vérifie toutes les **5 secondes** si des expéditions ont atteint leur destination. Si oui, le stock du magasin est mis à jour.

4. Concurrency

La production automatique (TD1) et le réapprovisionnement modifient le stock de l'usine en parallèle. Deux écritures concurrentes ne doivent jamais écraser silencieusement les données de l'autre. En cas de conflit, la commande doit être réessayée automatiquement (max 3 tentatives).

Voici des outils qui peuvent servir :

- <https://www.baeldung.com/jpa-optimistic-locking>
- <https://www.baeldung.com/spring-programmatic-transaction-management>



RIOC 2025

TD – Magasin / Usine - Geocodage

Alexandre Létocart

1. Géocodage des adresses

Les usines et magasins sont désormais créés avec une adresse. La base de données ne stocke que latitude et longitude — jamais l'adresse.

- Écriture : le client envoie une adresse → le serveur la convertit en coordonnées via l'API → persiste uniquement latitude et longitude
- Lecture : le serveur lit latitude et longitude depuis la base → appelle l'API inverse → retourne l'adresse dans la réponse

API de géocodage

ENDPOINT	URL	DESCRIPTION
Géocodage	GET /search/?q={adresse}&limit=1	Adresse → coordonnées
Inverse	GET /reverse/?lat={lat}&lon={lon}&limit=1	Coordonnées → adresse

Base URL : <https://api-gouv.lab.rioc.fr>

Pour tester, essayer sur un navigateur : <https://api-gouv.lab.rioc.fr/search/?q=quai%20de%20la%20somme>

Et : <https://api-gouv.lab.rioc.fr/reverse/?lat=49.904008&lon=2.286525>

- <https://www.baeldung.com/spring-boot-restclient>

- <https://www.baeldung.com/spring-value-annotation>

3. Distance Réelle

Remplacez la distance aléatoire par la formule de haversine. C'est la distance à vol d'oiseau entre deux points GPS.

Implémentez le calcul manuellement, sans bibliothèque externe.

- Le réapprovisionnement (POST /supply) utilise la vraie distance usine → magasin
- Les usines disponibles (GET /factories) affichent la vraie distance par rapport au magasin

- <https://www.baeldung.com/java-find-distance-between-points>

3. Endpoint de distance

Exposez un endpoint qui calcule la distance entre deux adresses quelconques. Cet endpoint sera utilisé dans le TP.

Contexte : Vous construisez un service e-commerce (**client-clipper**) qui permet à des clients d'acheter des trombones depuis les magasins gérés par l'API **Clipper** (Api créé en TD). Le service gère les comptes clients (inscription, connexion Basic Auth), les commandes avec 3 niveaux de livraison, et la livraison automatique en arrière-plan.

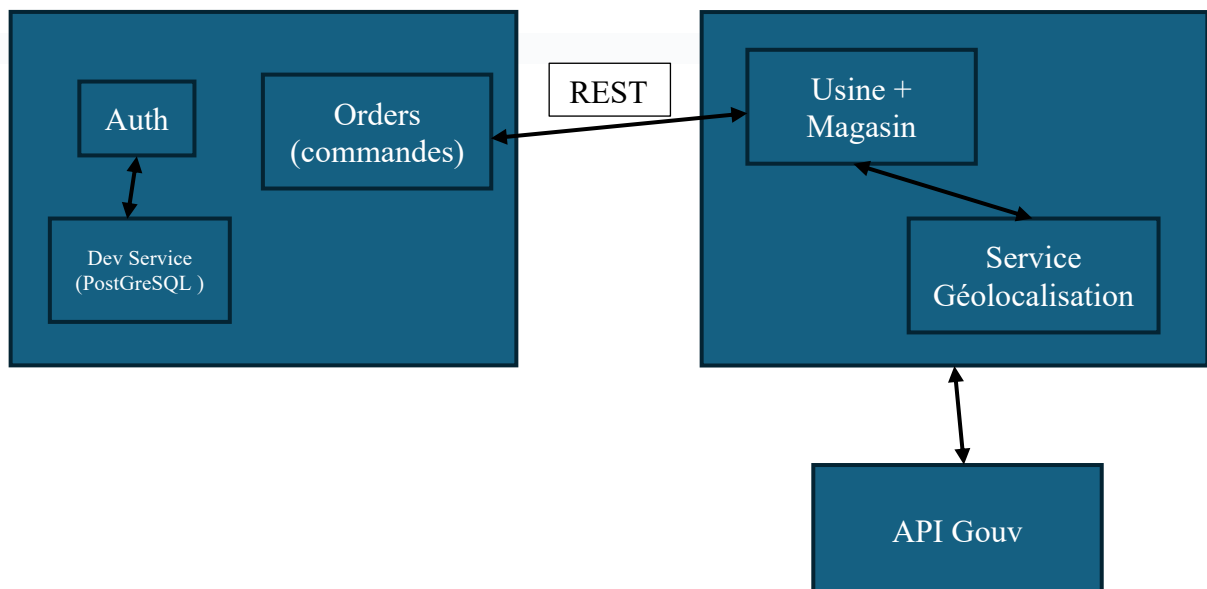
Stack : Java 21, Quarkus 3.39.2, PostgreSQL (Dev Services), Security JPA (Basic Auth)

Prérequis : TD Clipper terminé (Spring Boot sur le port 8080).

PS : Le code (noms de classes, variables, endpoints) doit être rédigé en anglais. Tous les DTO doivent être de records.

1. Architecture

Deux services communiquent par REST :



Principe fondamental : ne pas dupliquer ce que Clipper fait déjà. Clipper sait géocoder des adresses et calculer des distances — client-clipper délègue via des appels REST.

Pas besoin de docker-compose. Quarkus Dev Services détecte automatiquement les dépendances (ici PostgreSQL) et démarre les conteneurs Docker nécessaires au lancement de l'application en mode dev. Aucune configuration d'URL, d'utilisateur ou de mot de passe n'est requise.

Voici des liens qui peuvent servir pour comprendre l'architecture :

- <https://quarkus.io/guides/dev-services> (Dev Services — vue d'ensemble)
- <https://quarkus.io/guides/datasource#dev-services> (Dev Services PostgreSQL)
- <https://quarkus.io/guides/getting-started> (Premier projet Quarkus)
- <https://quarkus.io/guides/rest-client> (Communication inter-services via REST Client)

2. Configuration du projet

Générez un projet Quarkus via code.quarkus.io en sélectionnant les dépendances ci-dessous.

DÉPENDANCE	RÔLE
<code>quarkus-rest-jackson</code>	Endpoints REST + sérialisation JSON
<code>quarkus-rest-client-jackson</code>	Client REST typé pour appeler Clipper
<code>quarkus-hibernate-orm-panache</code>	ORM simplifié — Active Record (<code>PanacheEntity</code>)
<code>quarkus-jdbc-postgresql</code>	Driver PostgreSQL — Dev Services démarre le conteneur automatiquement
<code>quarkus-hibernate-validator</code>	Validation des DTOs (<code>@NotNull</code> , <code>@Positive</code> , <code>@Valid</code>)
<code>quarkus-security-jpa</code>	Basic Auth — vérifie les credentials automatiquement via l'entité JPA (<code>@UserDefinition</code>)
<code>quarkus-scheduler</code>	Jobs planifiés (<code>@Scheduled</code>)
<code>quarkus-smallrye-openapi</code>	Contrat d'interface — génère la spec OpenAPI et Swagger UI (<code>/q/swagger-ui</code>)

<http://localhost:8081/q/swagger-ui> -> Interface graphique pour explorer et tester tous les endpoints

Voici des liens qui peuvent servir :

- <https://quarkus.io/guides/rest>
- <https://quarkus.io/guides/rest-client>
- <https://quarkus.io/guides/hibernate-orm-panache>
- <https://quarkus.io/guides/datasource#dev-services>
- <https://quarkus.io/guides/validation>
- <https://quarkus.io/guides/security-jpa>
- <https://quarkus.io/guides/openapi-swaggerui> (OpenAPI + Swagger UI)

application.properties

```
# — Server —
quarkus.http.port=8081

# — REST Client : Clipper (Spring Boot sur le port 8080) —
quarkus.rest-client.clipper-api.url=http://localhost:8080

# — Hibernate ORM —
# Dev Services démarre PostgreSQL automatiquement, aucune URL/user/password
à configurer
quarkus.hibernate-orm.schema-management.strategy=drop-and-create

# — Sécurité : Basic Auth —
quarkus.http.auth.basic=true
```

3. Endpoint d'achat côté Clipper

Avant de continuer sur client-clipper, vous devez créer un endpoint dans le projet **Clipper** (Spring Boot) pour décrémenter le stock d'un magasin lors d'un achat.

MÉTHODE	URL	DESCRIPTION
POST	/api/stores/{id}/purchase	Acheter des trombones dans un magasin

4. Authentification Basic Auth

Implémentez un système d'inscription avec des mots de passe hashés en BCrypt. L'authentification est gérée automatiquement par Quarkus via **Basic Auth**, le client envoie ses credentials (email + mot de passe) à chaque requête et Quarkus vérifie le hash en base.

Création de l'entité Customer

L'annotation `@UserDefinition` indique à Quarkus que cette entité représente un utilisateur. Les annotations `@Username`, `@Password` et `@Roles` lui indiquent quels champs utiliser pour l'authentification.

Champs attendus		
CHAMP	TYPE	NOTES
<code>id</code>	Long	Clé primaire, héritée de <code>PanacheEntity</code>
<code>email</code>	String	Unique, not null — <code>@Username</code>
<code>password</code>	String	Hash BCrypt — <code>@Password</code> — jamais exposé dans les réponses
<code>role</code>	String	<code>@Roles</code> — défaut à <code>"user"</code>
<code>name</code>	String	Nom du client
<code>address</code>	String	Adresse postale complète (ex: "10 Rue de Rivoli 75004 Paris")
<code>balance</code>	BigDecimal	Solde du compte, défaut à 500.00
<code>paperclipStock</code>	Integer	Stock de trombones, défaut à 0

Service d'authentification

Créez un AuthService (@ApplicationScoped) avec deux méthodes :

- **register** — Hasher le mot de passe avec BcryptUtil.bcryptHash()
- **getProfile** — Retrouver un customer par email

MÉTHODE	URL	DESCRIPTION
POST	/api/auth/register	Inscription (@PermitAll), retourne 201
GET	/api/customers/{id}	Profil client (@RolesAllowed("user")), vérifie l'ownership

Protéger le profil

Le endpoint GET /api/customers/{id} doit vérifier que le client authentifié accède bien à **son propre** profil. Utilisez SecurityContext pour récupérer l'email du client authentifié, puis comparez l'{id} de l'URL avec l'id du customer trouvé en base :

```
@Context
SecurityContext securityContext;

@GET
@Path("/{id}")
public CustomerResponse getCustomer(@PathParam("id") Long id) {
    String email = securityContext.getUserPrincipal().getName();
    Customer customer = Customer.findByEmail(email);
    if (customer == null || !customer.id.equals(id)) {
        throw new ForbiddenException("You can only access your own profile");
    }
    return authService.getProfile(email);
}
```

Voici des liens qui peuvent servir :

- <https://quarkus.io/guides/security-jpa> (Security JPA — @UserDefinition, @Username, @Password, @Roles)
- <https://quarkus.io/guides/security-basic-authentication> (Basic Auth dans Quarkus)
- <https://quarkus.io/guides/hibernate-orm-panache>
- <https://quarkus.io/guides/security-getting-started-tutorial>
- <https://docs.oracle.com/en/java/javase/21/language/records.html> (Java Records)
- <https://www.baeldung.com/java-bcrypt> (comprendre BCrypt)
- <https://jakarta.ee/specifications/bean-validation/3.0/> (Jakarta Validation — @NotBlank, @Size, etc.)

5. Commandes et achat de trombones

Un client authentifié peut commander des trombones depuis un magasin Clipper. La commande inclut un type de livraison qui détermine la vitesse et le coût.

Clients REST vers Clipper

Pour passer des commandes, client-clipper doit appeler l'API Clipper (magasins, achat, distance). Créez deux interfaces annotées `@RegisterRestClient(configKey = "clipper-api")` :

- `ClipperStoreClient` — `@Path("/api/stores")` : lister les magasins, détail d'un magasin, achat
- `ClipperGeoClient` — `@Path("/api/geo")` : distance entre deux adresses

- <https://quarkus.io/guides/rest-client#create-the-interface>

Tarification

ÉLÉMENT	PRIX
Trombone (unité)	0.10 €
Livraison BASIC (100 km/h)	5.00 €
Livraison EXPRESS (300 km/h)	15.00 €
Livraison ULTRA (1 235 km/h — vitesse du son)	50.00 €

Prix total = (quantité × 0.10) + coût livraison

Les différents endpoints

Chaque endpoint doit retourner le code HTTP approprié.

MÉTHODE	URL	DESCRIPTION
POST	/api/orders	Passer une commande (body : storeId, quantity, deliveryType)
GET	/api/orders	Lister les commandes du client authentifié
GET	/api/orders/{id}	Détail d'une commande (vérifier l'ownership)
GET	/api/stores	Proxy sécurisé → Clipper GET /api/stores
GET	/api/stores/{id}	Proxy sécurisé → Clipper GET /api/stores/{id}

Tous ces endpoints nécessitent `@RolesAllowed("user")`.

Voici des liens qui peuvent servir :

- <https://quarkus.io/guides/rest-client#using-the-rest-client>
- <https://quarkus.io/guides/hibernate-orm-panache#transactions>
- <https://quarkus.io/guides/validation>
- <https://quarkus.io/guides/hibernate-orm-panache#most-useful-operations> (opérations Panache : list, find, persist)

6. Livraison automatique

Un processus en arrière-plan vérifie toutes les **5 secondes** si des commandes ont atteint leur destination. Si oui, le stock de trombones du client est mis à jour.

Vous devez créer un `DeliveryScheduler` (`@ApplicationScoped`) avec une méthode annotée `@Scheduled(every = "5s")` et `@Transactional` qui :

- Trouve les commandes en cours de livraison
- Change le status de livraison si la date de livraison est passé
- Ajoute la quantité au `paperclipStock` au bon client

Voici des outils qui peuvent servir pour cette automatisation :

- <https://quarkus.io/guides/scheduler>
- <https://quarkus.io/guides/scheduler#scheduled-methods>